

I Asked for May and Got August

Problem. A trade sync for 1 to 28 May returned trades from 10 August to 4 September. No error, no warning, and the trades it imported were real.

Root cause. The code measured the length of the requested range and passed that to a parameter that means "days back from today." A 31 day range became "the last 31 days."

Resolution. Fixed the calculation, added the date filter that was missing entirely, and found a third bug in the fix itself before it shipped.

Context

StrategyForge is a trading journal. Trades sync from a user's brokerage account through SnapTrade, an aggregator that sits between us and the broker.

The date range picker had been in the product for a while. I was testing a separate change and used it to pull a specific month, mostly out of habit.

The trades that came back were from the wrong months entirely. Not missing, not empty. Wrong.

Step 1. Confirm it is not the display

First question is always whether the data is wrong or the view is wrong. I queried the rows that had just been written:

```
SELECT DATE(entry_time) AS trade_date, COUNT(*) AS trades
FROM trades
WHERE created_at > NOW() - INTERVAL '1 hour'
GROUP BY 1 ORDER BY 1;
```

```
2026-08-10 | 1
2026-08-14 | 2
...
2026-09-04 | 7
```

Seventy-eight trades, dated 10 August through 4 September. I had asked for May. The wrong data was in the database, so this was an import problem rather than a filtering problem.

Step 2. Ask the API what it was told

I could have kept reading our own code. Instead I opened SnapTrade's dashboard, which logs every request we make to them, and looked at the call from that sync.

The query parameters:

```
{
  "days": ["31"],
  "state": ["all"],
  "userId": ["..."],
  "clientId": ["..."]
}
```

`days: 31` . No start date. No end date.

That single line explains everything. We never sent SnapTrade the dates at all. We sent the number 31.

I would not have found this from the code or the database. The sync reported success and imported genuine trades, just from a period nobody asked for.

Step 3. Where 31 came from

```
let days = 90;
if (startDate && endDate) {
  const start = new Date(startDate);
  const end = new Date(endDate);
  days = Math.ceil((end.getTime() - start.getTime()) / (1000 * 60 * 60 * 24));
}
```

It subtracts the start from the end. That is the **length** of the range.

The parameter it feeds means "this many days back from today." So a request for 1 to 28 May sent 31, and SnapTrade correctly returned the last 31 days.

Pick any seven day window and you get the last seven days. Pick a year and you get the last year. The picker was decorative, and it looked like it worked because something always came back.

The branch immediately below it, for a start date with no end date, does it correctly:

```
} else if (startDate) {
  const start = new Date(startDate);
  const now = new Date();
  days = Math.ceil((now.getTime() - start.getTime()) / (1000 * 60 * 60 * 24));
}
```

Same file, six lines apart, right the second time. Only the common case was wrong.

Step 4. The second defect

I assumed at this point that something downstream must be trimming the results to the requested dates, so the damage was limited. I checked instead of assuming.

```
const executedOrders = orders.filter((order) => {
  const status = order.status?.toUpperCase();
  return status === 'EXECUTED' || status === 'FILLED';
});
```

Status only. Nothing anywhere filtered by date.

So even with a correct lookback, everything from the start date to today would have imported. Two independent bugs, and either alone would have produced wrong results.

Step 5. The fix

The endpoint has no date parameters at all, only a lookback window. So the window has to be wide enough to contain the range, and the trimming has to happen on our side.

1. `days` now measures from today back to the start date, so the window contains the requested period.
2. A new filter drops anything outside the range before a trade is built from it.

Orders with no usable timestamp are kept rather than discarded. A missing field should not silently delete a real fill.

Step 6. A third bug, caught by the tests

I wrote twenty tests for the range logic. Nineteen passed. This one did not:

```
FAIL May 31 23:59:58 kept
```

The end of the range is meant to be inclusive, so a trade at 23:59:58 on the last day should be kept. It was being dropped.

```
if (rangeEnd) rangeEnd.setHours(23, 59, 59, 999);
```

`new Date('2026-05-31')` parses as **UTC** midnight. `setHours` sets **local** time. Mixing the two shifts the boundary by whatever the server's offset happens to be.

On my machine, which is UTC-6:

```
parsed as:      2026-05-31T00:00:00.000Z
after setHours: 2026-05-31T05:59:59.999Z
```

The last day of every range was being cut short by eighteen hours.

Changed to `setUTCHours`, which matches how the date was parsed in the first place. The tests now run under three timezones so the bug cannot come back:

```
--- Denver (UTC-6) ---    20 passed, 0 failed
--- UTC ---                20 passed, 0 failed
--- Tokyo (UTC+9) ---     20 passed, 0 failed
```

This one would have hidden in production indefinitely. Railway runs its containers in UTC, where the offset is zero and the bug does not appear. It would only ever have shown up for someone running the server somewhere else.

Step 7. Deploy, and a limit I could not fix

Re-ran the same May sync on staging. The log now read:

```
[SNAPTRADE] Fetching CACHED orders for account Robinhood Individual (130 days)
[SNAPTRADE] Processing 0 total orders
[SNAPTRADE] Date range 2026-05-01 → 2026-05-28: kept 0 of 0 executed orders
```

130 days, correctly reaching back to May. Zero orders returned.

From SnapTrade's SDK documentation for that parameter:

*Number of days in the past to fetch the most recent orders. Defaults to the last 30 days if no value is passed in.
Values greater than 90 will be capped at 90.*

May was 127 days ago. The cap is 90. No calculation on our side could reach it.

Sending 130 does not error. It silently returns the last 90 days, our new filter correctly discards all of it, and the user sees "0 trades imported" with no way to tell a broker limitation from a broken sync.

I capped the value at 90 deliberately so we stop making a request we know will be trimmed, and the sync response now carries a flag the UI can use to explain an empty result.

Step 8. The data was there the whole time

The same sync showed this a few lines earlier:

```
[SNAPTRADE] Fetched 1 activities from account Robinhood Crypto
  "units": -0.001379, "ticker": "BTC", "trade_date": "2026-05-12T01:09:09Z"
[SNAPTRADE] Fetched 46 activities from account Robinhood Individual
```

Zero orders and forty seven activities, same accounts, same period.

SnapTrade has two endpoints. Orders carry exact execution times and reach back a few months. Activities go back years, sometimes to account inception, but refresh once daily. We were building trades from orders only and using activities purely to look up prices.

Full history was never going to work from the orders endpoint. That is a design problem rather than a bug, and it is the next piece of work.

What I'd take from this

Ask the API what it received. The bug was invisible in our code and invisible in our database. It took eight seconds to find once I read SnapTrade's request log, because the outgoing parameters said `days: 31` and no dates. Vendor-side logs are underused.

A feature that always returns something looks like it works. Nobody reported this. The picker returned trades every time, and they were real trades, so there was nothing obvious to complain about.

Correct code six lines away is not reassuring. The version handling a start date with no end date was right. The common path was wrong. Adjacent correctness meant nothing.

Verify the assumption you are about to rely on. I assumed a downstream filter existed and nearly stopped there. Checking took a minute and found a second bug that would have survived the first fix.

Write the test even when the logic looks obvious. The timezone bug was in code I had just written to fix another bug, it looked correct, and it would never have appeared on the production server. The test caught it because it ran somewhere with a non-zero offset.

Know the vendor's limits before promising the feature. The 90 day cap is in SnapTrade's own documentation. Reading it earlier would have saved the fix that could not work.

Log output is from a real broker sync on a test account. Identifiers removed.